JJBike

1. Objective.

Build a data warehouse so that the administrator of the fictional company JJBike can understand their business.

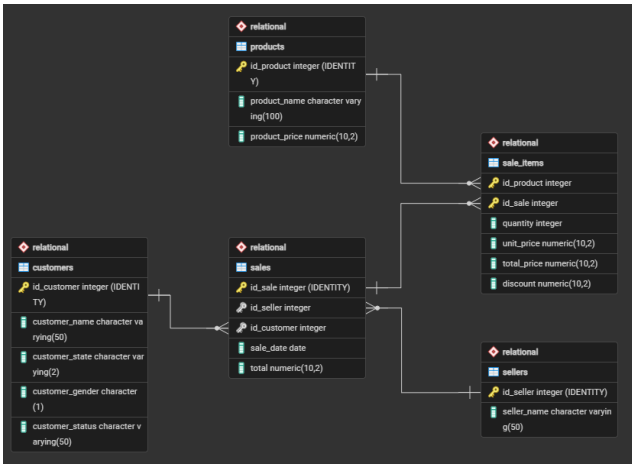
2. Modeling.

To create the analytical database structure model (OLAP) considering the transactional database (OLTP), the subject to be analyzed (the fact), and the dimensions (the various viewpoints from which to analyze the fact (the dimensions)), using the Star Schema model (the most common model for Business Intelligence), the following transformations were performed:

Dimension	Attributes	Source Table
dim_product (what?)	product_key (SK)	-
	id_product	product
	product_name	product
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_seller (who?)	seller_key (SK)	-
	id_seller	seller
	seller_name	seller
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_customer (for whom?)	customer_key (SK)	-
	id_customer	customer
	customer_name	customer
	customer_state	customer
	customer_gender	customer
	customer_status	customer
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_time (when?)	time_key (SK)	-

	date	-
	time_day	-
	time_month	-
	time_year	-
	time_weekday	-
	time_quarter	-

Fact	Consolidated Table	Granularity	Attributes	Source Table
fact_sales	sales	Item sold.	sale_key (SK)	-
			seller_key (FK)	dim_seller
			customer_key (FK)	dim_customer
			product_key	dim_product
			time_key (FK)	dim_time
			quantity	sale_items
			unit_price	sale_items
			total_price	sale_items
			discount	sale_items



3. Transactional environment (OLTP) - Microsoft SQL Server Management Studio (SSMS).

3.1. Creation of the relational schema.

The Relational database was created, serving as the source layer for the transactional data of the JJBike company. Within it, the file 1. Scripts_Create_Table_Relational.sql created five operational tables. In all

of them, the primary key is automatically generated by the IDENTITY clause, eliminating the need to control external sequences. All columns, except the primary keys themselves, were defined with NOT NULL, ensuring that no record is persisted with mandatory data open.

- **sellers:** Stores the sellers' data. The seller_name attribute identifies the seller's name and is the only field besides the PK, reflecting the intentional scope of this entity in the transactional model;
- **products:** Stores the product catalog. The product_name attribute identifies the product and product_price records the current list price at the time of registration. Both are mandatory, as a product without a name or price cannot participate in a sales process; customers: Stores customer registration information. The customer_name attribute identifies the customer; customer_state records the state of origin (Varchar(2)); customer_gender records the gender in a single character (Char(1)); and customer_status records the customer's plan level, an attribute that, because it changes over time, will be the main element tracked by SCD Type 2 in the analytical environment;
- **sales:** Represents the header of each sale. The id_seller attribute is a foreign key that references Relational..sellers, and id_customer is a foreign key that references Relational..customers, both mandatory, as a sale cannot exist without an associated seller and customer. The sale_date attribute records the date on which the transaction occurred with type DATETIME, and total stores the consolidated value of the sale;
- **sale_items:** Linking table between sales and products, with a primary key composed of id_product and id_sale. Referential integrity constraints were declared via ALTER TABLE after table creation:
    - FK_SALEITEMS_PRODUCT in id_product: references Relational..products, preventing historical data from losing the reference to the transacted product;
    - FK_SALEITEMS_SALE in id_sale: references Relational..sales, ensuring that items are always associated with an existing sale;
    - The attributes quantity, unit_price, total_price, and discount are the detail metrics for each item and were declared NOT NULL because they will directly populate the measure columns of the fact table in the analytical environment.

3.2. Table Population.

The data was inserted using the files 2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql, and 6. Insert_Into_SaleItems.sql. Each file executes INSERT INTO commands on the respective table in the Relational database, specifying only the data columns, primary keys are omitted because the IDENTITY mechanism assigns them automatically and sequentially to each inserted record.

4. Staging environment (invisible/conceptual).

There was no treatment or cleaning of raw data before creating the dimensions; the direct data transfer between PostgreSQL schemas itself acted as the load trigger.

5. Analytical environment (OLAP) - Microsoft SQL Server Management Studio (SSMS).

5.1. Creation of the database and dimensional tables.

The Dimensional database was created, representing the Data Warehouse structured in a Star Schema model. The file 1. Scripts_Create_Table_Dimensional.sql created the dimensions dim_seller, dim_customer, dim_product, and dim_time, in addition to the fact table fact_sales. In all tables, the Surrogate Key (SK) is automatically generated by IDENTITY, serving as the internal primary key of the DW, decoupled from the source IDs of the transactional system.

All columns were defined with NOT NULL, with the sole exception of validity_end_date in dimensions with SCD Type 2. This field remains NULL while the record is active; it is only filled with the current time when an attribute change is detected and a new version of the record needs to be created. The validity fields are of type DATETIME, allowing the differentiation of records processed on the same date with millisecond precision.

Dimensional tables have the following attribute structure:

- **dim_seller:** seller_key is the automatically generated SK and acts as the internal PK of the DW. id_seller preserves the source ID from Relational..sellers, maintaining traceability between the two environments. seller_name carries the seller's name. validity_start_date receives GETDATE() when the record is inserted, marking the start of its validity. validity_end_date remains NULL while the record is the active version;
- **dim_customer:** customer_key is the SK, the internal PK of the DW. id_customer preserves the source ID. customer_name, customer_state, customer_gender, and customer_status are the tracked descriptive attributes, any change in any of them triggers SCD Type 2 versioning. validity_start_date and validity_end_date control the validity period of each version of the record;
- **dim_product:** product_key is the SK. id_product preserves the source ID. product_name is the only descriptive attribute tracked, a change in its value triggers versioning. validity_start_date and validity_end_date function the same way as in other SCD Type 2 dimensions;
- **dim_time:** time_key is the SK. time_date stores the complete date and serves as a linking field with the transactional system during fact loading. time_day, time_month, time_year, and time_quarter are derived from time_date and enable analytical groupings by temporal granularity. time_weekday stores the day of the week as an integer (1 = Sunday, 7 = Saturday), following the SQL Server DATEPART(weekday, ...) pattern. This dimension does not have validity fields because time is immutable;
- **fact_sales:** sale_key is the SK of the fact. seller_key, customer_key, product_key, and time_key are the foreign keys (FKs) that reference their respective dimensions, all declared NOT NULL, since each fact must be associated with all dimensions of the model. quantity, unit_price, total_price, and discount are the metrics for each item sold, also NOT NULL.

5.2. Populating the time dimension.

The dim_time dimension was populated by the file 2. Insert_Into_dim_time.sql. The strategy adopted was the programmatic generation of a continuous date range from January 1, 1970 to December 31, 2030. The script consists of three chained blocks:

- **Date format configuration:** SET DATEFORMAT dmy defines the interpretation of dates in the day/month/year pattern for the session, ensuring that literal dates are read correctly by SQL Server;

- **Recursive CTE date_sequence:** uses a CTE anchored at @start_date and recurses via DATEADD(day, 1, generated_date) until it reaches @end_date. The OPTION (MAXRECURSION 0) clause removes the default 100 iteration limit of SQL Server, allowing the recursion to traverse all days of the interval without interruption;
- **INSERT final:** inserts each generated date into dim_time, extracting time_day via DAY(), time_month via MONTH(), time_year via YEAR(), time_weekday via DATEPART(weekday, ...) and time_quarter via DATEPART(quarter, ...).

5.3. Extraction (E) and loading (L) of data from the transactional environment (OLTP) to the analytical environment (OLAP).

The loads were performed using the 3.Script.sql file, structured in sequential phases to demonstrate the complete operation of the Type 2 SCD.

5.3.1. Initial Loading of Dimensions.

The same MERGE pattern with OUTPUT was applied to the three dimensions supporting SCD Type 2. Before each loading block, two temporal control variables are declared: @dt_end captures the exact moment of execution via GETDATE() and is used to close the old record by filling in validity_end_date; @dt_start receives @dt_end + 3 milliseconds via DATEADD(MILLISECOND, 3, @dt_end) and is used as the validity_start_date of the new record, ensuring chronological order even when both operations occur in the same execution. The mechanism operates in two chained steps:

- **MERGE:** compares the Relational database (source) with the destination dimension in the Dimensional database. The join condition T.id_X = S.id_X AND T.validity_end_date IS NULL locates, for each source ID, the only record still active in the dimension. The WHEN MATCHED AND block (attributes diverge) executes UPDATE SET T.validity_end_date = @dt_end, ending the validity of that version. The WHEN NOT MATCHED BY TARGET block directly inserts completely new records. The OUTPUT $ACTION clause outputs a row for each operation performed, including the action performed ('UPDATE' or 'INSERT') and the source attributes;
- **External INSERT:** wraps the MERGE in a subquery and filters only the rows where action = 'UPDATE', that is, the records that had a version ended. For these records, it inserts a new active row in the dimension with validity_start_date = @dt_start and validity_end_date = NULL, preserving the complete version history. New records ('INSERT') are ignored at this stage because they have already been handled directly by the MERGE.

5.3.2. Loading the fact table (month by month).

The fact_sales table is populated by an INSERT INTO ... SELECT that consolidates data from the Relational database with the dimensions of the Dimensional database. Sales were loaded month by month, January, February, March, and subsequent months, to enable the simulation and verification of SCD Type 2 between loads.

The SELECT block operates with the following attribute and key mapping:

- **Metrics:** quantity, unit_price, total_price, and discount are extracted directly from Relational..sale_items, as they represent the detail data of each item sold, the granularity of the fact table;

- **seller_key:** Obtained by the INNER JOIN between Relational..sales and Dimensional..dim_seller using v.id_seller = vdd.id_seller AND vdd.validity_end_date IS NULL. The validity_end_date IS NULL filter ensures that the fact table records the SK of the seller version that was active at the time of loading;
- **customer_key:** Obtained by INNER JOIN between Relational..sales and Dimensional..dim_customer using v.id_customer = c.id_customer AND c.validity_end_date IS NULL. The validity_end_date IS NULL filter is the central mechanism of SCD Type 2 in fact loading: it ensures that the registered SK points to the current customer_status at the time that month was loaded;
- **product_key:** Obtained by INNER JOIN between Relational..sale_items and Dimensional..dim_product using iv.id_product = p.id_product AND p.validity_end_date IS NULL. The same filter captures only the active version of the product at the time of loading;
- **time_key:** Obtained by INNER JOIN between Relational..sales and Dimensional..dim_time using v.sale_date = t.time_date. The sale_date field from the transactional table is directly compared to the time_date field of the dimension, locating the record that represents exactly the day the sale occurred;
- **Temporal filter:** the MONTH(v.sale_date) function extracts the month number from sale_date and restricts the load to sales from the desired period, = 1 for January, = 2 for February, = 3 for March, and > 3 for the remaining months.

5.3.3. Simulation of status change and history processing.

To demonstrate SCD Type 2 versioning, two direct updates were applied to Relational..customers:

- **After the January load:** UPDATE Relational..customers SET customer_status = 'Gold' WHERE id_customer BETWEEN 1 AND 5. The customer_status attribute of customers with IDs 1 to 5 was promoted to 'Gold';
- **After the March load:** UPDATE Relational..customers SET customer_status = 'Platinum' WHERE id_customer = 3. The customer_status attribute of the customer with ID 3 was promoted to 'Platinum'.

With each change, the load block of dimensions (MERGE + OUTPUT + INSERT) was re-executed. The mechanism detects the discrepancy in customer_status, closes the previous record by filling validity_end_date with @dt_end and inserts a new line with validity_start_date = @dt_start and validity_end_date = NULL, preserving the complete version history of each customer.

5.3.4. Verification and Audit of SCD Type 2.

Throughout the roadmap, audit queries were executed to validate the behavior of SCD Type 2. The SELECTs cross fact_sales with dim_customer, and when necessary with dim_time, via INNER JOIN by the respective SKs, filtering c.id_customer BETWEEN 1 AND 5. The points below show what the queries demonstrate:

- The January records in fact_sales point to the old SKs of customers 1 to 5, that is, to the version in which customer_status was not yet 'Gold'. The historical link is preserved in the fact even after the update in the dimension;
- The records from February onwards point to the new SKs, reflecting the customer_status in effect at the time each month was loaded;

- An additional query with the filter t.time_month = 3 confirms that none of the clients with IDs 1 to 5 made purchases in March, validating that the fact did not generate records for this group during that period.

5.4. KPI (Key Performance Indicator).

To measure JJBike's revenue against monthly targets, the Dimensional..kpi table was created using the 4.KPI.sql file. The script consists of four main structures:

- **Preliminary cleanup:** the IF OBJECT_ID('Dimensional..kpi', 'U') IS NOT NULL DROP TABLE Dimensional..kpi block checks if the table already exists before recreating it, avoiding errors in script re-executions;
- **SELECT block:** defines the three columns that will make up the table: dt.time_month aliased as month, which identifies the reference month; SUM(fs.total_price) aliased as total_revenue, which aggregates the revenue actually earned by summing the total_price of all fact_sales records for the respective month; and a CASE dt.time_month block that assigns a fixed target value for each month, from R$ 220,000 in January to R$ 270,000 in December, aliased as target;
- **Type conversion:** the CAST(... AS NUMERIC(18,2)) operator positioned after the END of the CASE ensures that the target column is consolidated with the same fixed-precision numeric type as total_price in fact_sales. Without this conversion, SQL Server would treat the CASE values as integers, causing type incompatibility when calculating achievement percentages;
- The FROM block starts with Dimensional..fact_sales and performs an INNER JOIN with Dimensional..dim_time based on the condition fs.time_key = dt.time_key, relating each fact record to its respective month. The GROUP BY dt.time_month consolidates all sales for each month into a single row, and the ORDER BY dt.time_month ASC organizes the result in ascending chronological order. The SELECT INTO operator physically creates the Dimensional..kpi table from the query result.

6. Artifacts - Power BI.

The tables from the Dimensional database, created in SSMS, were connected to Power BI.

6.1. Reports.

6.1.1. Transformations (T) in the tables.

In fact_sales, a column called discount_percent (with two decimal places) was created with the following formula:

- discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2);

In dim_customer, a column called location_map was created with the following formula:

- location_map =
  SWITCH(
  'dimensional dim_customer'[customer_state],
  "AC", "Acre",

```
"AL", "Alagoas",
"AM", "Amazonas",
"AP", "Amapa",
"BA", "Bahia",
"CE", "Ceara",
"DF", "Federal District",
"ES", "Espirito Santo",
"GO", "Goias",
"MA", "Maranhao",
"MG", "Minas Gerais",
"MS", "Mato Grosso do Sul",
"MT", "Mato Grosso",
"PA", "Para",
"PB", "Paraiba",
"PE", "Pernambuco",
"PI", "Piaui",
"PR", "Parana",
"RJ", "Rio de Janeiro",
"RN", "Rio Grande do Norte",
"RO", "Rondonia",
"RR", "Roraima",
"RS", "Rio Grande do Sul",
"SC", "Santa Catarina",
"SE", "Sergipe",
"SP", "Sao Paulo",
"TO", "Tocantins",
"Unknown"
) & ", Brazil".
```

6.1.2. Metrics.

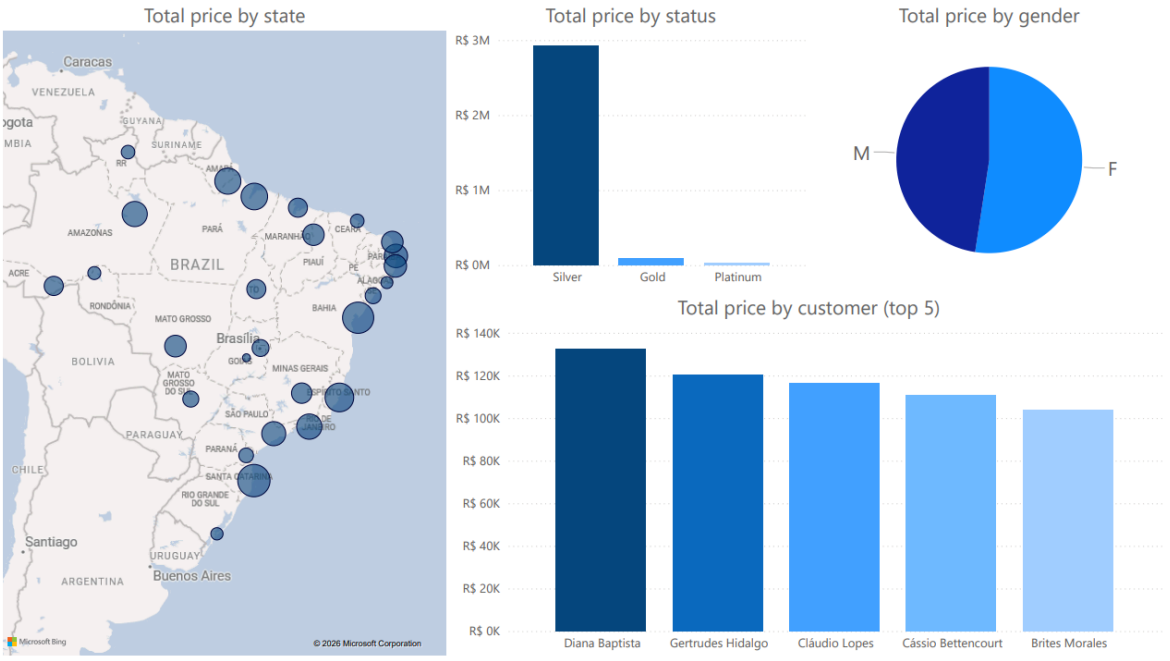
In dim_kpi, three metrics were created so that, in the KPI report, the card has dynamic behavior according to what is selected in the button slicer:

- measure_target =
  ```
  IF(
      ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
      SUM('dimensional kpi'[target]),
      0
  ).
  ```
- measure_total_revenue =
  ```
  IF(
      ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
      SUM('dimensional kpi'[total_revenue]),
      0
  ).
  ```

6.1.3. Created Reports.

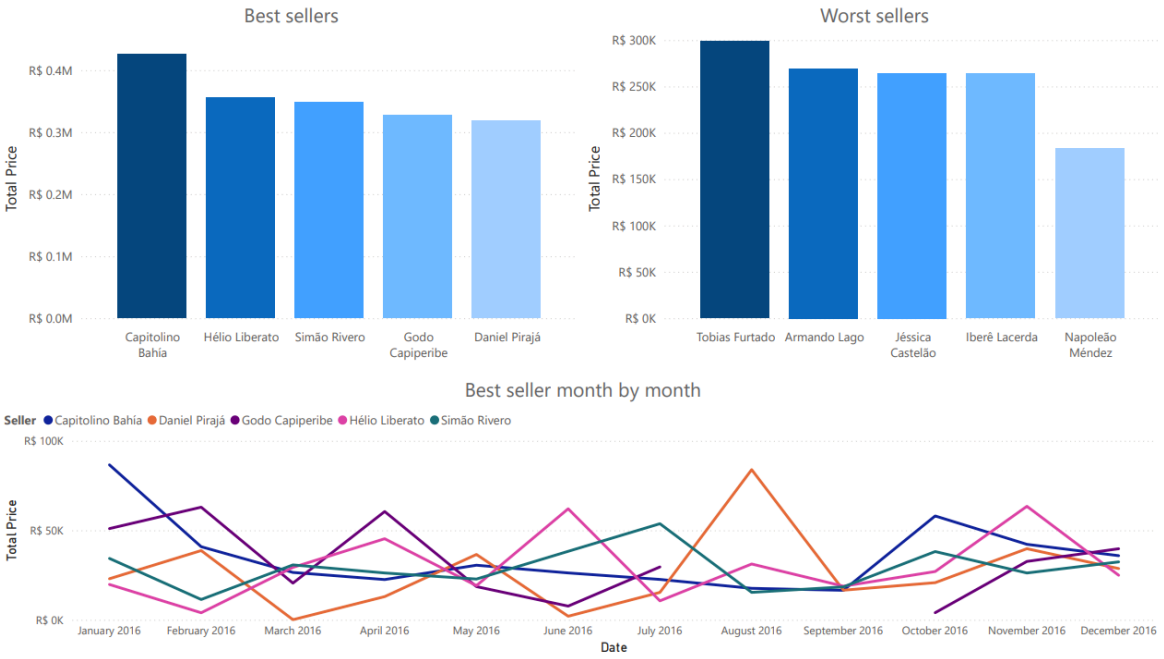
(Customers)

The customer report shows: total value sum by state, total value sum by status, total value sum by gender, and total value sum by customer (top 5).



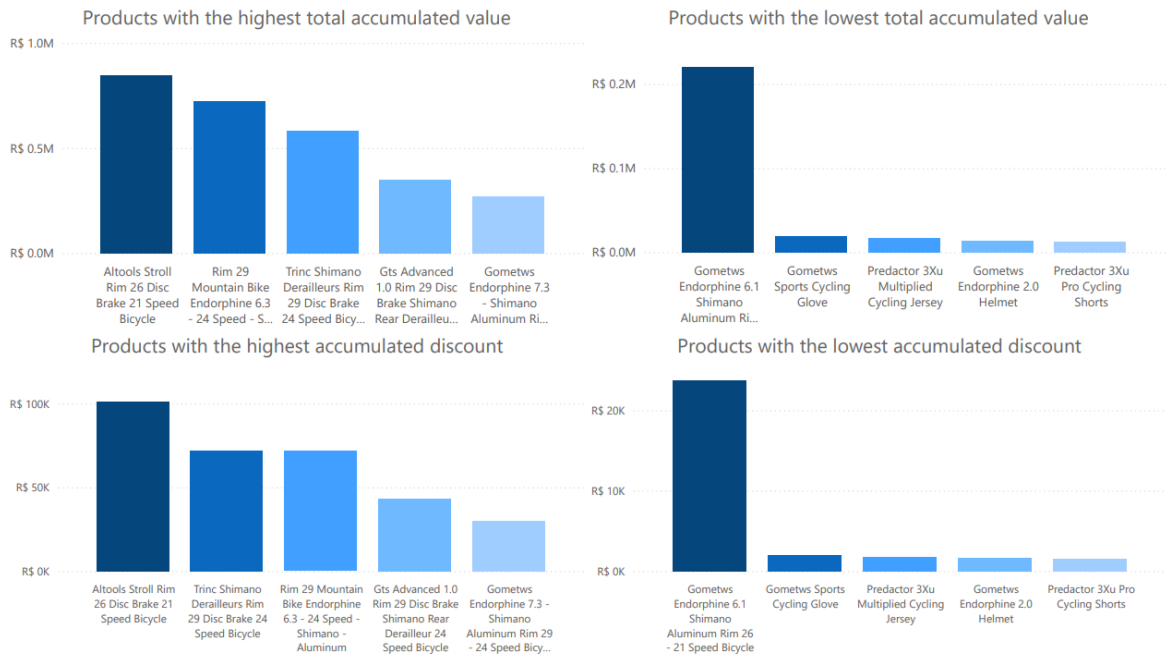
(Sellers)

The sellers report shows: total value sum by sellers (top 5 best sellers, top 5 worst sellers, and top 5 best sellers compared month by month).



(Products)

The product report shows: total value per product (top 5 best-selling products and top 5 least-selling products) and total discount per product (top 5 products with the highest accumulated discount and top 5 products with the lowest accumulated discount).



6.2. Dashboards.

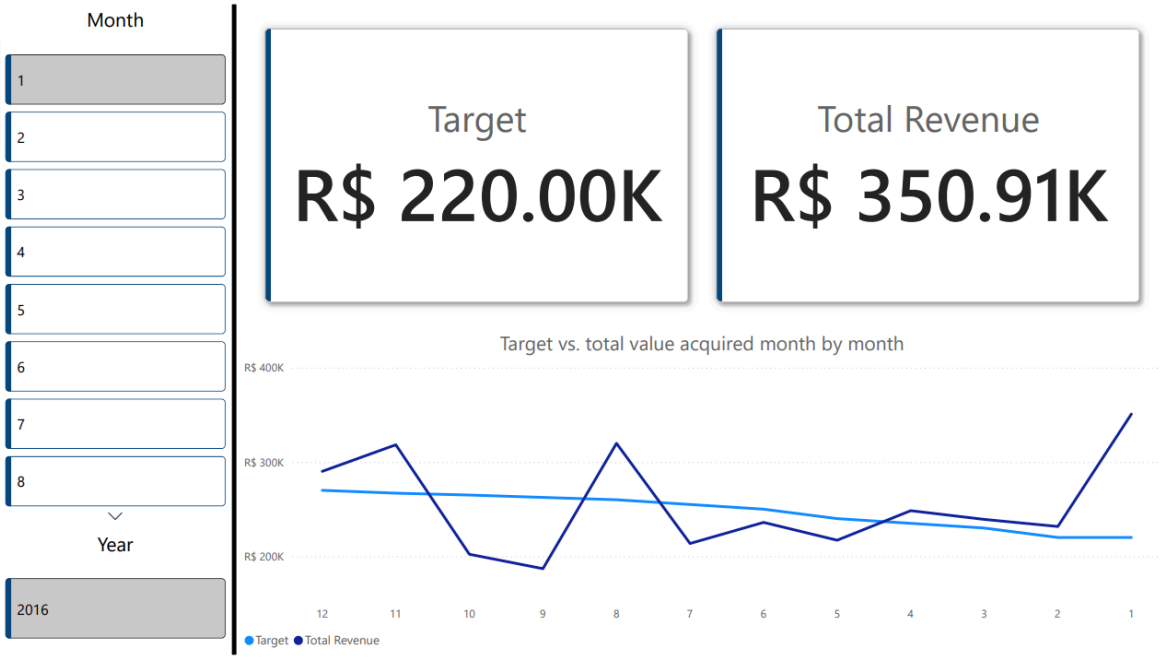
The following dashboards were created:

(Sales)

The sales report shows: total value of products sold (compared month by month), total quantity of products sold (compared month by month), total discount of products sold (compared month by month), and product, sellers, and customer cards for filtering.

(KPI)

The KPI report shows: target, total value acquired, sum of target and total value acquired (compared month by month), and month and year cards for filtering.



Generated file is available in '2. Artifacts/1. Reports+Dashboards/'.

7. Artifact - Microsoft Visual Studio.

7.1. Cube.

The Dimensional database tables were structured and hosted on a full instance of SQL Server (Developer Edition), ensuring native support for Business Intelligence (BI) functionalities;

A Data Source View (DSV) was created, a visual environment where the fact and dimension tables were mapped (excluding the kpi table), generating a logical model in Star Schema format;

The following dimensions were created:

- **dim_customer:** customer_key, customer_name, customer_gender, customer_state, and customer_status;
- **dim_product:** product_key and product_name;
- **dim_time:** time_key, time_year, time_quarter, time_month, and time_day;
- **dim_seller:** seller_key and seller_name.

The following hierarchies were created from the dimensions:

- **Geographic Hierarchy:** customer_state → customer_name (dim_customer).
  - **Note:** In the Attribute Relationships tab, the order should be: Customer Key → Customer Name → Customer State.
- **Temporal Hierarchy:** time_year → time_quarter → time_month → time_day (dim_time).

    - **Note:** In the Attribute Relationships tab, the order should be: Time Key → Time Day → Time Month → Time Quarter → Time Year.
- **Observations:**
    - customer_gender and customer_status (from dim_customer) should remain free in the side panel as Non-hierarchical attributes, as inserting them within Geographic Hierarchy would imply that they are part of a geographic subdivision;
    - dim_seller and dim_product have only one attribute in addition to their keys, therefore they did not generate hierarchies, and their attributes became Non-hierarchical attributes;
    - To avoid duplicate key errors in the Analysis Services engine during dim_time processing, the KeyColumns properties were customized as composite keys, linking time_quarter with time_year, time_month with time_year, and time_day with time_month and time_year, ensuring the uniqueness and integrity of the attribute relationship tree.

The cube was structured from the fact_sales fact table. During the process, relationship keys and analytically incorrect fields (such as unit_price, whose aggregation by sum makes no business sense) were ignored. Only the actual performance metrics were retained (quantity, total_price, discount, and automatic record counting);

A successful deployment of the project was performed on the local instance of the Analysis Services server, compiling the structures and loading the data into memory;

Visual representation of the created cube:

Customer State	Customer Name	Time Year	Time Quarter	Time Month	Time Day	Quantity	Discount	Total Price
AC	Alarico Quinterno	2016	4	12	10	1	60.45	155
AC	Antônio Sobral	2016	3	9	17	1	1593.36	8852
AC	Antônio Sobral	2016	4	10	16	2	1741.25	7935.6
AC	Basilio Soares	2016	1	3	29	1	4.65	155
AC	Basilio Soares	2016	2	4	17	9	879.08	22167.65
AC	Basilio Soares	2016	2	4	18	3	125.1	3289
AC	Basilio Soares	2016	2	5	14	1	64.07	2135.52
AC	Basilio Soares	2016	2	6	6	1	650.93	6509.3
AC	Basilio Soares	2016	2	6	24	1	16.92	188
AC	Basilio Soares	2016	3	7	5	2	775.25	7793
AC	Carla Briones	2016	4	12	5	1	587.75	2260.58
AC	Ester Castanho	2016	3	9	26	5	2327.33	15416.98
AC	Evaristo Bahía	2016	4	10	10	3	692.17	4836.7
AC	Irene Villanueva	2016	4	11	12	3	1654.34	7183.75
AL	Cid Pardo	2016	3	8	21	3	1769.02	13295
AL	Cid Pardo	2016	4	10	18	3	2430.31	13522.61
AM	Deise Farias	2016	3	8	22	3	1231.04	8976.8
AM	Deise Farias	2016	3	9	7	1	1472.16	9201
AM	Diana Baptista	2016	1	2	12	4	79.67	11112.58
AM	Diana Baptista	2016	1	2	13	1	1.69	169.2
AM	Diana Baptista	2016	1	3	15	4	285.75	7205.75
AM	Diana Baptista	2016	2	4	2	1	56.31	5631.01

Generated files are available in '2. Artifacts/2. Cube/'.